
Telegram AI Bot

Technical Documentation & Developer Reference

Version 1.0 | Built on AWS EC2

Groq API • Pollinations AI • Python-Telegram-Bot

1. Project Overview

This project is a Telegram chatbot that provides two AI-powered capabilities to its users: real-time conversational AI chat via the Groq API, and text-to-image generation via Pollinations AI. The bot is hosted on an AWS EC2 virtual server and runs as a continuously active Python process.

Project Type	Telegram Bot — AI Chat + Image Generation
Hosting	AWS EC2 (Linux virtual server)
Language	Python 3 (async)
Chat AI	Groq API — llama-3.3-70b-versatile model
Image AI	Pollinations AI — free image generation
Bot Framework	python-telegram-bot v20+
HTTP Client	httpx (async)

1.1 Core Capabilities

- Multi-turn AI conversation using Groq-hosted Llama 3.3 70B
- Per-user conversation memory (in-memory, last 10 messages)
- Text-to-image generation via the /image command
- Live typing indicator so users know the bot is working
- Session reset via the /start command

2. System Architecture

The bot uses a long-polling model: it continuously asks Telegram 'are there any new messages?' and processes them one at a time. No inbound ports need to be open on the server. All communication is initiated outward from the EC2 instance to Telegram's servers.

```

[Telegram User]
  |
  | text message / command
  v
[AWS EC2 Instance] — bot.py (Python, async)
  |                     |
  v                     v
[Groq API]             [Pollinations AI]
LLM Chat Reply         Image Generation
  |                     |
  v                     v
[Telegram User receives reply or generated photo]

```

2.1 Chat Request Flow

- User types a message and sends it
- Telegram forwards it to the bot via long polling
- Bot appends it to the user's in-memory history
- Last 10 messages are sent to Groq API as context
- Groq returns an AI response — bot saves it to history and sends it back

2.2 Image Request Flow

- User sends /image followed by a description
- Bot URL-encodes the description and builds a Pollinations URL
- The URL is passed to Telegram's reply_photo method
- Telegram fetches and displays the image — the bot never downloads it

3. Configuration & Environment

All secrets and service endpoints are defined as constants near the top of bot.py. For personal projects this is acceptable, but for shared or open-source code these should always be moved to environment variables.

```

# Current hardcoded approach (fine for personal use)
TELEGRAM_TOKEN = "***..."
GROQ_API_KEY   = "***..."
GROQ_URL       = "https://api.groq.com/openai/v1/chat/completions"
MODEL          = "llama-3.3-70b-versatile"

```

Security Note: Never commit real API keys to GitHub or share them in public code. If your keys are exposed, regenerate them immediately in the Groq Console and BotFather.

3.1 Recommended — Use Environment Variables

Environment variables let you store secrets outside your code. Here is how to update the configuration block:

```
import os

TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN")
GROQ_API_KEY    = os.getenv("GROQ_API_KEY")
```

Then set them on the EC2 server before running the bot:

```
export TELEGRAM_TOKEN=your_token_here
export GROQ_API_KEY=your_key_here
python3 bot.py
```

4. Dependencies

Package	Purpose
python-telegram-bot	Async Telegram Bot API wrapper (v20+)
httpx	Async HTTP client — used to call the Groq API
logging	Built-in Python logging — no install needed
os	Built-in — reads environment variables

4.1 Installation

```
pip install python-telegram-bot httpx
```

5. Command & Handler Reference

5.1 /start

Resets the user's conversation history and sends a welcome message with usage instructions.

Trigger	/start
Effect	Clears in-memory history for that user
Response	Welcome message with feature overview

5.2 /image <prompt>

Generates a 1024x1024 image from a text description using the Pollinations AI free API.

Trigger	/image <prompt text>
Image Size	1024 x 1024 pixels
Provider	Pollinations AI (free, no auth required)
Method	URL passed to Telegram — Telegram fetches it directly
Timeout	Handled by Telegram; no server-side download needed

Bug in original code: `encoded_prompt` and `image_url` were computed after the `try/except` block that uses them, causing a `NameError`. See Section 7 for the fix.

5.3 Text Message Handler

Catches all non-command messages and responds using Groq's LLM.

Trigger	Any plain text message (not a command)
Model	llama-3.3-70b-versatile (Groq)
Context Window	Last 10 messages per user
API Timeout	40 seconds
Typing Indicator	Shown while waiting for Groq response

6. State Management

The bot keeps conversation context in a plain Python dictionary called `user_histories`. There is no database. Everything lives in the process memory of the running bot.

```
user_histories = {}

# Grows to look like this as users chat:
# {
#   87654321: [
#     {'role': 'user',      'content': 'What is Python?'},
#     {'role': 'assistant', 'content': 'Python is ...'},
#     {'role': 'user',      'content': 'Give me an example'},
#   ],
#   11223344: [ ... another user ... ]
# }
```

All history is lost when the bot stops or the EC2 instance reboots. For persistent memory across restarts, a database (SQLite, Redis, or DynamoDB) would need to be added.

Behaviour	Detail
Isolation	Each user's history is completely separate
Context sent	Last 10 messages only (older ones are dropped)
Reset trigger	/start command or process restart
Memory growth	Unbounded — no automatic cleanup between sessions

7. Known Issues & Bug Fixes

7.1 Variable Scope Bug in image_gen

In the original file, `encoded_prompt` and `image_url` are computed after the `try/except` block that references them. Python evaluates code top-to-bottom, so the variables do not exist yet when they are needed, and the function will crash with a `NameError` every time `/image` is used.

Fix: move the variable definitions before the `try` block.

```
# WRONG — variables used before they are defined
try:
    await update.message.reply_photo(photo=image_url ...) # NameError!
except ...
encoded_prompt = httpx.utils.quote(prompt) # too late

# CORRECT — define first, then use
encoded_prompt = httpx.utils.quote(prompt)
image_url = f'https://pollinations.ai/prompt/{encoded_prompt}...'
try:
    await update.message.reply_photo(photo=image_url ...)
except ...
```

7.2 Duplicate Logic in image_gen

The original function has two separate `try/except` blocks and two different URL constructions. One downloads the image via `httpx` (complex, slow), and the other passes the URL directly to Telegram (simpler, faster). Only one is needed. The URL pass-through approach is recommended because it saves server bandwidth and avoids 60-second download timeouts on the bot side.

7.3 Missing Empty Prompt Guard

If a user sends `/image` with nothing after it, `context.args` will be an empty list. Joining an empty list produces an empty string, and the image API will receive a blank prompt. A guard should be added at the top of the function:

```
if not context.args:
    await update.message.reply_text('Usage: /image a sunset over Lagos')
```

```
return
```

8. Deployment Guide (AWS EC2)

This section walks through every step required to get the bot running on a cloud server, from creating the server to keeping the bot alive permanently. Each step is explained in plain language — no prior cloud experience assumed.

8.1 What is EC2 and Why Use It?

Amazon EC2 (Elastic Compute Cloud) is a service that lets you rent a virtual computer in the cloud. Think of it as a PC that lives in Amazon's data centre and is always switched on. Your bot.py file runs on this machine 24/7, even when your own laptop is off.

You connect to it over the internet using a tool called SSH (Secure Shell) — the same way you would remote-control another computer. Once connected, it looks and works like a normal Linux terminal.

8.2 Step 1 — Create an AWS Account & Launch an EC2 Instance

If you do not already have an AWS account, create one at aws.amazon.com. You will need a credit card, but the free tier covers a small instance (t2.micro or t3.micro) for 12 months.

Launching the instance:

- Log in to the AWS Console at console.aws.amazon.com
- Search for EC2 in the top search bar and click it
- Click the orange Launch Instance button
- Give your instance a name, e.g. telegram-bot-server
- Under Application and OS Images, choose Ubuntu Server 22.04 LTS (free tier eligible)
- Under Instance type, select t2.micro (free tier) or t3.micro
- Under Key pair, click Create new key pair — name it telegrambot-key and download the .pem file. This file is your password to the server. Store it safely — you cannot recover it.
- Under Network settings, leave the defaults. The bot only makes outbound connections, so no inbound ports need to be opened (SSH port 22 is open by default for your management access)
- Click Launch Instance

The .pem key file can only be downloaded once. If you lose it you will need to create a new key pair and reassign it to the instance. Keep it in a safe folder on your machine.

8.3 Step 2 — Connect to the Server via SSH

SSH lets you open a command-line session on your remote EC2 instance from your local terminal (Mac/Linux) or PowerShell (Windows). You use the .pem key file to authenticate.

On Mac or Linux:

```
# First, restrict permissions on the key file (required by SSH)
chmod 400 ~/Downloads/telegrambot-key.pem

# Connect - replace YOUR_EC2_PUBLIC_IP with your instance's IP
# (Find it in EC2 > Instances > your instance > Public IPv4 address)
ssh -i ~/Downloads/telegrambot-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

On Windows (PowerShell):

```
# Windows does not need chmod, just run:
ssh -i C:\Users\You\Downloads\telegrambot-key.pem ubuntu@YOUR_EC2_PUBLIC_IP
```

After connecting you will see a prompt like `ubuntu@ip-172-xx-xx-xx:~$` — you are now inside the cloud server.

Use `ubuntu@` as the username for Ubuntu instances, or `ec2-user@` for Amazon Linux instances. Using the wrong username is a common cause of 'Permission denied' errors on first connection.

8.4 Step 3 — Install Python and Dependencies

Ubuntu 22.04 comes with Python 3 pre-installed. You just need to install the two bot dependencies.

```
# Update the package list (good practice before installing anything)
sudo apt update

# Install pip if it is not already present
sudo apt install python3-pip -y

# Install the bot's two Python dependencies
pip3 install python-telegram-bot httpx

# Verify the install worked
python3 -c "import telegram, httpx; print('Dependencies OK!')"
```

8.5 Step 4 — Upload bot.py to the Server

You need to copy your `bot.py` file from your local machine to the EC2 server. The `scp` (Secure Copy) command does this over the same SSH connection.

From your local machine terminal (not the SSH session):

```
# Run this on your own laptop, not inside the EC2 SSH session
scp -i ~/Downloads/telegrambot-key.pem \
/path/to/your/bot.py \
ubuntu@YOUR_EC2_PUBLIC_IP:~/bot.py
```

This copies bot.py into the home directory (/home/ubuntu/) on the server. To confirm it arrived:

```
# Back inside your SSH session on EC2:
ls ~
# You should see: bot.py
```

Alternatively, you can paste the code directly on the server using nano bot.py, then pasting your code, pressing Ctrl+X, Y, and Enter to save.

8.6 Step 5 — Test the Bot Manually

Before setting up any background service, confirm the bot works by running it directly. This is the fastest way to catch errors.

```
# Inside your EC2 SSH session:
python3 bot.py

# Expected output:
# INFO - Application started
# Bot started. Model: llama-3.3-70b-versatile
```

Open Telegram, find your bot, and send it a message. If it replies, the bot is working correctly. Press Ctrl+C to stop it — you will set it up as a background service in the next step.

If you see 'Conflict: terminated by other getUpdates request', another instance of the bot is already running somewhere. Make sure no other copies are active (check your local machine or other SSH sessions).

8.7 Step 6 — Run the Bot as a Permanent Background Service

By default, when you close your SSH terminal the bot stops. To keep it running permanently — and restart it automatically if it crashes or the server reboots — you register it as a systemd service.

systemd is the process manager built into Linux. It handles starting, stopping, and monitoring long-running programs. Think of it like Task Scheduler on Windows or launchd on Mac, but more powerful.

Step 6a: Create the service file

This file tells systemd how to run your bot. Use nano to create it:

```
sudo nano /etc/systemd/system/telegrambot.service
```

Paste the following content into the file exactly as shown:

```
[Unit]
Description=Telegram AI Bot
After=network.target
```

```
[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu
ExecStart=/usr/bin/python3 /home/ubuntu/bot.py
Restart=always
RestartSec=10
StandardOutput=journal
StandardError=journal

[Install]
WantedBy=multi-user.target
```

Save and exit: press Ctrl+X, then Y, then Enter.

What each section means:

- [Unit] — describes what this service is and when it should start (after networking is ready)
- [Service] — defines how to run it: which user, which directory, which command, and to always restart on failure
- [Install] — makes the service start automatically on every boot

Step 6b: Enable and start the service

```
# Tell systemd to re-read service files (run after any .service file change)
sudo systemctl daemon-reload

# Enable the service so it starts automatically on every reboot
sudo systemctl enable telegrambot

# Start it now (without needing to reboot)
sudo systemctl start telegrambot

# Check that it is running
sudo systemctl status telegrambot
```

A healthy status output looks like this:

```
● telegrambot.service - Telegram AI Bot
   Loaded: loaded (/etc/systemd/system/telegrambot.service; enabled)
   Active: active (running) since Wed 2026-05-20 10:14:32 UTC
   Main PID: 1234 (python3)
```

If the status shows 'failed' or 'inactive', check the logs with: `sudo journalctl -u telegrambot -n 50` — this prints the last 50 lines of bot output and usually reveals the error.

8.8 Useful Management Commands

Once the bot is running as a service, these are the commands you will use most often:

What you want to do	Command
---------------------	---------

Check if bot is running	<code>sudo systemctl status telegrambot</code>
Stop the bot	<code>sudo systemctl stop telegrambot</code>
Restart the bot	<code>sudo systemctl restart telegrambot</code>
View recent logs	<code>sudo journalctl -u telegrambot -n 100</code>
Follow live logs	<code>sudo journalctl -u telegrambot -f</code>
Disable auto-start on boot	<code>sudo systemctl disable telegrambot</code>
Update bot.py and restart	<code>scp ... then sudo systemctl restart telegrambot</code>

8.9 Updating the Bot

When you make changes to bot.py locally, you need to re-upload the file and restart the service. Here is the full update workflow:

```
# 1. From your local machine – upload the updated file
scp -i ~/Downloads/telegrambot-key.pem bot.py ubuntu@YOUR_EC2_IP:~/bot.py

# 2. SSH into the server
ssh -i ~/Downloads/telegrambot-key.pem ubuntu@YOUR_EC2_IP

# 3. Restart the service to load the new code
sudo systemctl restart telegrambot

# 4. Confirm it came back up cleanly
sudo systemctl status telegrambot
```

8.10 Troubleshooting Common Problems

Problem	Likely Cause & Fix
Bot not responding in Telegram	Check status: <code>systemctl status telegrambot</code> . Check logs: <code>journalctl -u telegrambot -n 50</code>
'Permission denied' on SSH	Wrong username (use ubuntu@) or wrong .pem file path. Check <code>chmod 400</code> on the key.
'Conflict: terminated by other request'	Two bot instances running. Stop all and start fresh with <code>systemctl</code> .
'ModuleNotFoundError: telegram'	Dependencies not installed for the right Python. Run: <code>pip3 install python-telegram-bot httpx</code>
Bot crashes after a few minutes	Check logs for error. Usually an API key issue or network problem. Service will auto-restart in 10s.
scp: No such file	Wrong local path to bot.py. Make sure you are in the right folder on your machine.

9. Future Improvements

Area	Suggestion
Security	Move API keys to environment variables or AWS Secrets Manager
Persistence	Add Redis or DynamoDB to preserve chat history across restarts
Memory management	Add LRU eviction to user_histories to prevent unbounded growth
Error handling	Add retry logic with exponential backoff for Groq API failures
System prompt	Add a configurable system prompt to give the bot a persona
Rate limiting	Throttle requests per user to prevent API quota exhaustion
Webhook mode	Switch from polling to webhook for lower latency
Monitoring	Ship logs to CloudWatch or set up uptime alerts
Image options	Support aspect ratio flags and /image alias

10. Quick Reference

Item	Value
Start command	/start — resets session
Image command	/image <prompt> — 1024x1024 image
Chat	Send any plain text message
Context window	Last 10 messages per user
Groq model	llama-3.3-70b-versatile
Image provider	pollinations.ai
Bot framework	python-telegram-bot v20+
Main file	bot.py
Run manually	python3 bot.py
Service name	telegrambot (systemd)
Service start	sudo systemctl start telegrambot
View logs	sudo journalctl -u telegrambot -f

Appendix A — Full Corrected bot.py

The listing below is a clean, fully corrected version of the bot that resolves all issues described in Section 7. Comments have been added throughout to explain each block of code for future maintainers.

Replace the placeholder strings for TELEGRAM_TOKEN and GROQ_API_KEY with your real values, or set them as environment variables on the server (see Section 3.1).

```
import logging
import httpx
import os
from telegram import Update
from telegram.ext import (
    ApplicationBuilder,
    MessageHandler,
    CommandHandler,
    ContextTypes,
    filters
)

# — CONFIGURATION —
# Recommended: load from environment variables instead of
# hardcoding. Set them on the server before running:
# export TELEGRAM_TOKEN=your_token
# export GROQ_API_KEY=your_key

TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN", "YOUR_TOKEN_HERE")
GROQ_API_KEY = os.getenv("GROQ_API_KEY", "YOUR_KEY_HERE")
GROQ_URL = "https://api.groq.com/openai/v1/chat/completions"
MODEL = "llama-3.3-70b-versatile"

# — IN-MEMORY HISTORY —
# Stores conversation context per Telegram user_id.
# Cleared on /start or when the process restarts.
user_histories = {}

# — LOGGING —
logging.basicConfig(
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s',
    level=logging.INFO
)

# — /start COMMAND —
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Resets conversation history and greets the user."""
    user_id = update.effective_user.id
    user_histories[user_id] = []
    await update.message.reply_text(
        "🌀 *AI Assistant Active*\n\n"
        "• Send me a message to chat (Powered by Groq)\n"
        "• Use `/image [prompt]` to generate art (Pollinations)",
        parse_mode='Markdown'
    )

# — /image COMMAND —
async def image_gen(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Generates an image from the user's prompt via Pollinations AI."""
```

```

# Guard: make sure the user actually provided a prompt
if not context.args:
    await update.message.reply_text(
        "Please provide a prompt.\nUsage: /image a sunset over Lagos"
    )
    return

prompt = ' '.join(context.args)

# Encode the prompt so it is safe to embed in a URL
encoded_prompt = httpx.utils.quote(prompt)
image_url = (
    f"https://pollinations.ai/prompt/{encoded_prompt}"
    f"?width=1024&height=1024&nologo=true"
)

try:
    # Pass the URL directly to Telegram – Telegram fetches it.
    # This avoids downloading the image on the bot server.
    await update.message.reply_photo(
        photo=image_url,
        caption=f'🌀 *Prompt:* {prompt}',
        parse_mode='Markdown'
    )
except Exception as e:
    logging.error(f'Image Error: {e}')
    await update.message.reply_text(
        "⚠️ Telegram had trouble fetching that image. Try a different prompt."
    )

# ——— TEXT MESSAGE HANDLER ———
async def handle_message(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """Handles plain text messages and replies using the Groq LLM API."""
    user_id = update.effective_user.id
    user_text = update.message.text

    # Initialise history for first-time users
    if user_id not in user_histories:
        user_histories[user_id] = []

    # Append user message to their history
    user_histories[user_id].append({"role": "user", "content": user_text})

    # Keep only the last 10 messages (sliding window context)
    history = user_histories[user_id][-10:]

    # Show typing indicator while waiting for Groq
    await update.message.chat.send_action("typing")

    try:
        headers = {
            "Authorization": f"Bearer {GROQ_API_KEY}",
            "Content-Type": "application/json"
        }
        payload = {
            "model": MODEL,
            "messages": history,
            "stream": False
        }

```

```

    async with httpx.AsyncClient(timeout=40.0) as client:
        response = await client.post(GROQ_URL, headers=headers, json=payload)

    if response.status_code != 200:
        logging.error(f'Groq API Error: {response.text}')
        await update.message.reply_text(
            "⚠ Groq API is currently unavailable. Try again shortly."
        )
        return

    data = response.json()
    reply = data['choices'][0]['message']['content']

    # Save assistant reply to maintain conversation context
    user_histories[user_id].append({
        "role": "assistant",
        "content": reply
    })

    await update.message.reply_text(reply)

except Exception as e:
    logging.error(f'Chat Error: {e}')
    await update.message.reply_text(
        "⚠ Something went wrong. Please try again later."
    )

# ——— ENTRY POINT ———
if __name__ == "__main__":
    app = ApplicationBuilder().token(TELEGRAM_TOKEN).build()

    # Register commands BEFORE the general message handler
    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("image", image_gen))

    # Catch all plain text (non-command) messages
    app.add_handler(
        MessageHandler(filters.TEXT & ~filters.COMMAND, handle_message)
    )

    print(f'Bot started. Model: {MODEL}')
    app.run_polling()

```

Appendix B — External API Reference

B.1 Groq API

Endpoint	https://api.groq.com/openai/v1/chat/completions
Auth	Bearer token in Authorization header
Model used	llama-3.3-70b-versatile

Request format	OpenAI-compatible JSON (model, messages, stream)
Docs	console.groq.com/docs
Free tier	Yes — generous rate limits for personal projects

Example request payload sent by the bot:

```
{
  "model": "llama-3.3-70b-versatile",
  "stream": false,
  "messages": [
    {"role": "user", "content": "What is the capital of France?"},
    {"role": "assistant", "content": "The capital of France is Paris."},
    {"role": "user", "content": "Tell me more about it."}
  ]
}
```

B.2 Pollinations AI Image API

Endpoint	https://pollinations.ai/prompt/{encoded_prompt}
Auth	None required — completely free and open
Parameters	width, height, seed, nologo
Response	Redirect to a generated JPEG image
Docs	pollinations.ai
Limitations	Prompts are public; no adult content filtering applied

Example URL constructed by the bot:

```
https://pollinations.ai/prompt/a%20sunset%20over%20Lagos
?width=1024&height=1024&nologo=true
```

B.3 Telegram Bot API

Bot creation	Talk to @BotFather on Telegram — /newbot command
Token location	Sent by BotFather after bot creation
Polling method	<code>run_polling()</code> — bot asks Telegram for updates every few seconds
Webhook method	Alternative: bot receives updates via HTTPS endpoint (not used here)
Docs	core.telegram.org/bots/api